L03 – Preprocessing in Pandas

Data Mining
Dr Andrea Visentin

Thanks to Dr Alejandro Arbelaez

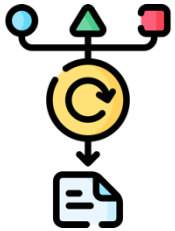
Data Preprocessing Intro

Forms of data preprocessing

- Data Cleaning

- Data Integration

- Data Transformation



- Data Reduction

Data Cleaning

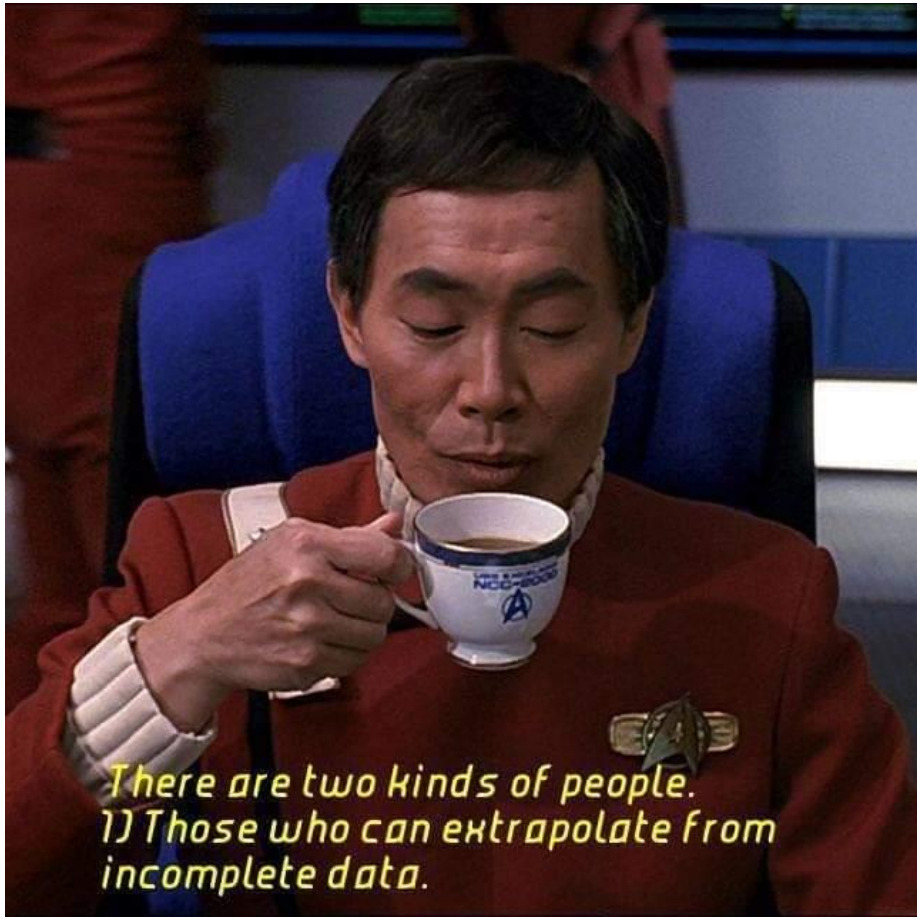
Data cleaning tasks

- Fill in missing values
- Identify outliers and smooth out noisy data
- Correct inconsistent data

Missing Data

- Data is not always available
  - Many tuples have no recorded value for several attributes, such as customers income in sales data
- Missing data may be due to
  - Equipment malfunction
  - Inconsistent with other recorded data and thus deleted
  - Data not entered due to misunderstanding
  - Certain data may not be considered important at the time of entry
  - Not register history of the changes
- Missing data may need to be **inferred**

Missing Data

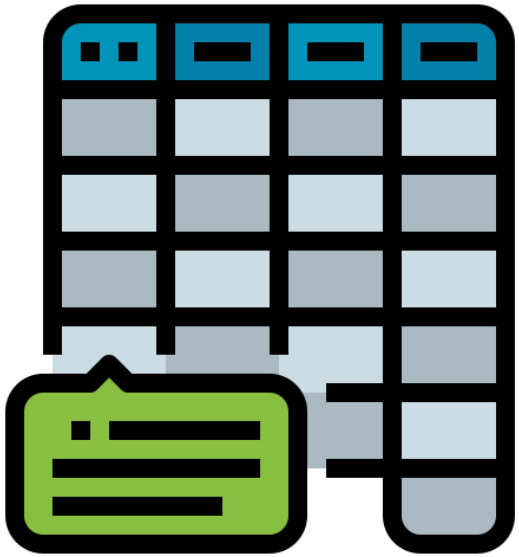


Missing Data: what to do?

- **Ignore the tuple**: usually done when class label is missing (assuming the task in classification – not effective when the percentage of missing values per attribute varies considerably)
- **Fill in the missing value manually**: tedious + infeasible?
- **Use a global constant to fill in the missing value**: e.g., "unknown", a new class?
- **Use the attribute mean to fill in the missing value**
- **Use the most probable value to fill the missing value**: inference-based such as Bayesian formula or decision tree

Summarizing Data

- With lots of playtesting, there will be a lot of data
  - This is a good thing!
- But raw data is just a pile of numbers
  - Rarely of interest
  - Or even sensible
- Q: How to summarize all this information?

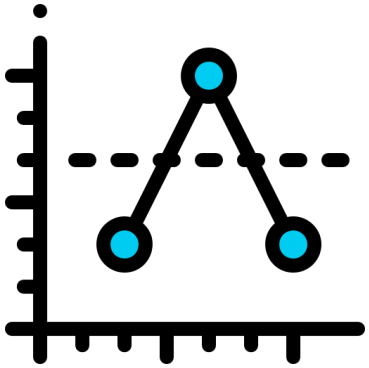


Measure of Central Tendency

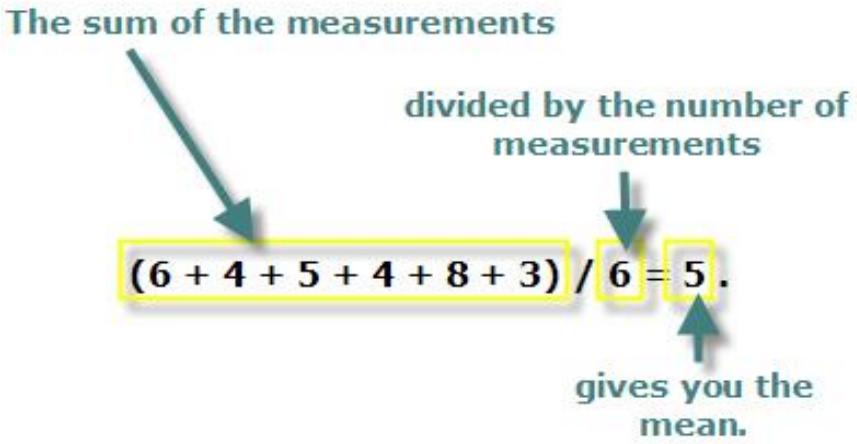
A **measure of central tendency** is a single value that attempts to describe a set of data by identifying the central position within that set of data.

As:

- Mean
- Median
- Mode



Mean



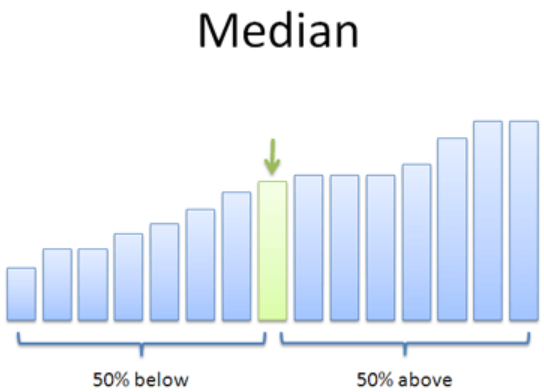
http://www.cdn.sciencebuddies.org/Files/463/9/MeanEquation.jpg

- Also called the **arithmetic mean** or average
- In Python:

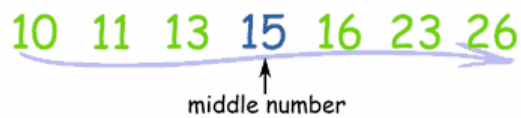
```
sum(my_list)/len(my_list)
```

Median

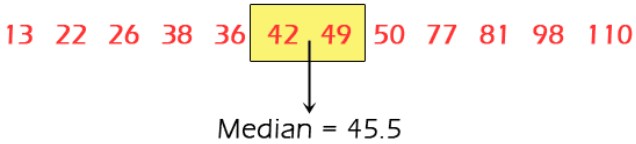
- Sort values low to high and take middle value



https://betterexplained.com/wp-content/uploads/average/median.png



https://www.mathsisfun.com/definitions/images/median.gif



http://www.nedarc.org/statisticalHelp/basicStatistics/measuresOfCenter/images/median.gif

- In Python:

```
import statistics
statistics.median(my_list)
```

Mode

- Mode: most common (frequent) number
- Peak of distribution
- Always an observed score

A, A, A, A, B, B, C, D, E

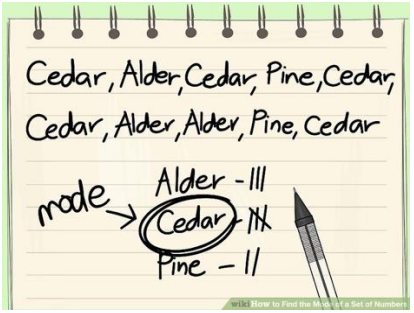
Mode

- Mode: most common (frequent) number
- Peak of distribution
- Always an observed score

$$\underbrace{\textbf{A, A, A, A,}}_{\textbf{Mode}}\ \text{B, B, C, D, E}$$

Mode

- Number which occurs most frequently
- Not so useful in many cases
- Best use for categorical data
  - e.g., most played champion in League

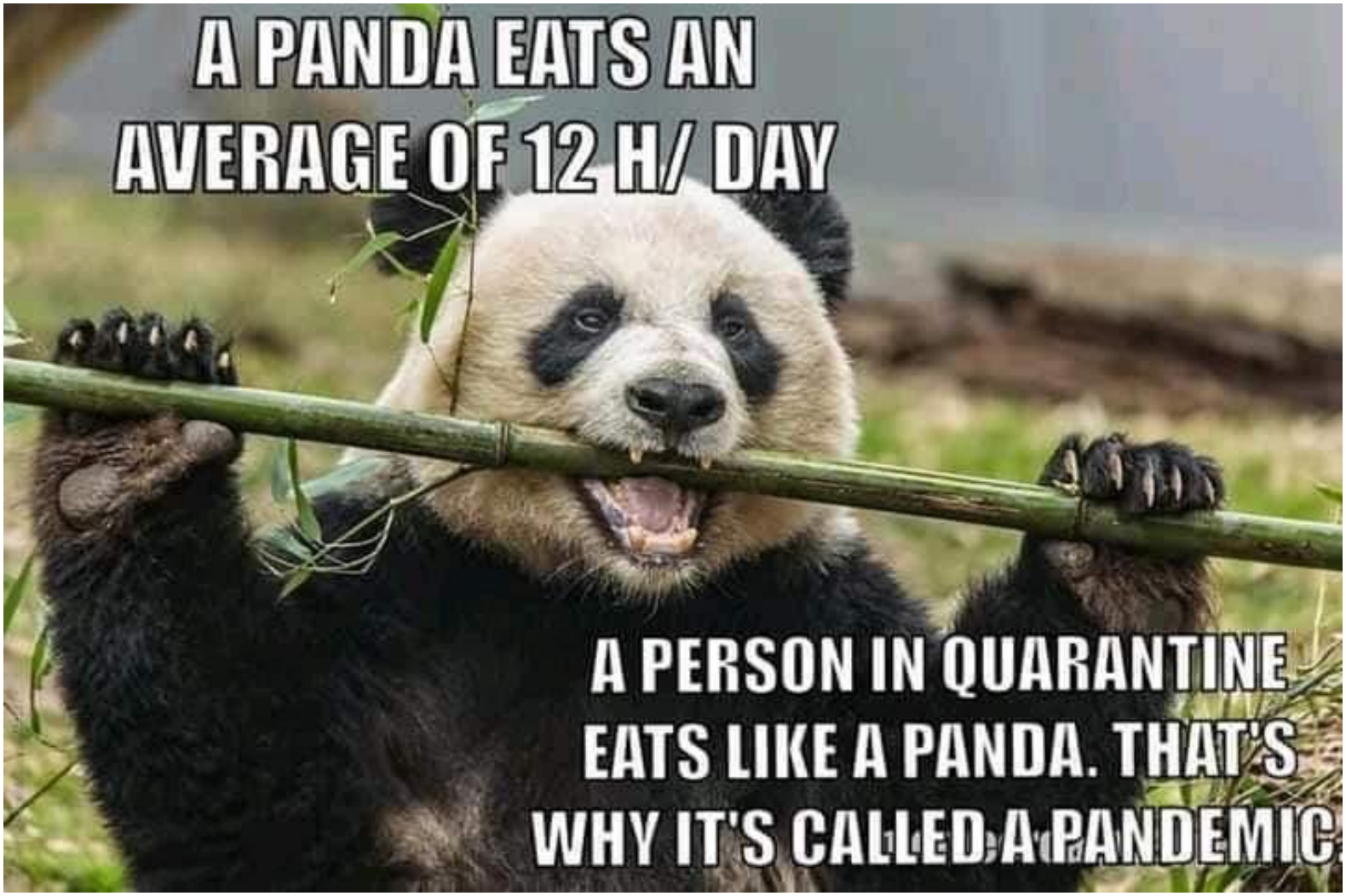


http://pad3.whstatic.com/images/thumb/c/cd/Find-the-Mode-of-a-Set-of-Numbers-Step-7.jpg/aid130521-v4-728px-Find-the-Mode-of-a-Set-of-Numbers-Step-7.jpg

- In Python:

```
import statistics
statistics.mode(my_list)
```

Let’s go back to Pandas


A PANDA EATS AN
AVERAGE OF 12 H/ DAY
A PERSON IN QUARANTINE
EATS LIKE A PANDA. THAT'S
WHY IT'S CALLED A PANDEMIC

Selecting a column in a Data Frame

Method 1: Subset the data frame using column name:

- `df[‘Age']`

Method 2: Use the column name as an attribute:

- `df.Age`

Data Frames methods

Unlike attributes, python methods have parenthesis.

They can have parameters.

- All attributes and methods can be listed with a dir() function:
  - `dir(df)`

Data Frames methods

df.method()	Description
head([n]), tail([n])	first/last n rows
describe()	generate descriptive statistics (for numeric columns only)
max(), min()	return max/min values for all numeric columns
mean(), median()	return mean/median values for all numeric columns
std()	standard deviation
sample([n])	returns a random sample of the data frame
dropna()	drop all the records with missing values

DataFrame - Head

DataFrame.head()

- returns the first rows
- you can select how many rows

DataFrame - Head

main.py

```python
import pandas as pd

# Import the data from the csv file
df = pd.read_csv("titanic.csv")

# This prints the first rows of the df
print(df.head())
```

Console Shell

```
   PassengerId  Survived  ...  Parents/Children Aboard     Fare
0            1         0  ...                      0.0   7.2500
1            2         1  ...                      0.0  71.2833
2            3         1  ...                      0.0   7.9250
3            4         1  ...                      NaN  53.1000
4            5         0  ...                      0.0   8.0500

[5 rows x 9 columns]
```

DataFrame - Head

main.py

```python
import pandas as pd

# Import the data from the csv file
df = pd.read_csv("titanic.csv")

# This prints the first 8 rows of the df
print(df.head(8))
```

Console Shell

```
   PassengerId  Survived  ...  Parents/Children Aboard     Fare
0            1         0  ...                      0.0   7.2500
1            2         1  ...                      0.0  71.2833
2            3         1  ...                      0.0   7.9250
3            4         1  ...                      NaN  53.1000
4            5         0  ...                      0.0   8.0500
5            6         0  ...                      0.0   8.4583
6            7         0  ...                      0.0  51.8625
7            8         0  ...                      1.0  21.0750

[8 rows x 9 columns]
```

DataFrame - Tail

main.py ×

```python
import pandas as pd

# Import the data from the csv file
df = pd.read_csv("titanic.csv")

# This prints the last 3 rows of the df
print(df.tail(3))
```

Console Shell

```
 PassengerId  Survived  ...  Parents/Children Aboard   Fare
884         885         0  ...                      2.0  23.45
885         886         1  ...                      0.0  30.00
886         887         0  ...                      0.0   7.75

[3 rows x 9 columns]
```

DataFrame - Describe

main.py ×

```
import pandas as pd

# Import the data from the csv file
df = pd.read_csv("titanic.csv")

# Describe the Age column
print(df.Age.describe())
```

Console Shell

```
count    887.000000
mean      29.471443
std       14.121908
min        0.420000
25%       20.250000
50%       28.000000
75%       38.000000
max       80.000000
Name: Age, dtype: float64
```

DataFrame – Max/Min

main.py ×

```
import pandas as pd

# Import the data from the csv file
df = pd.read_csv("titanic.csv")

# Display the min age
print(df.Age.min())

# Display the max fare
print(df["Fare"].max())
```

Console Shell

```
0.42
512.3292
```

Hands-on exercises

- ✓Give the summary for the numeric columns in the dataset
- ✓Calculate the percentage of people that survived,
- ✓What are the mean values of the first 50 records in the dataset?
  - ✓Hint: use head() method to subset the first 50 records and then calculate the mean

Missing Values

Inserting missing values

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	-1.2
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	-1.5
five	2002	Nevada	2.9	-1.7

Missing values

Missing Values

- There are a number of methods to deal with missing values in the data frame:

df.method()	description
dropna()	Drop missing observations
dropna(how='all')	Drop observations where all cells is NA
dropna(axis=1, how='all')	Drop column if all the values are missing
dropna(thresh = 5)	Drop rows that contain less than 5 non-missing values
fillna(0)	Replace missing values with zeros
isnull()	returns True if the value is missing
notnull()	Returns True for non-missing values

Filtering out missing data

main.py

```
import pandas as pd
from numpy import nan as NA

data = pd.DataFrame(
    [[1., 6.5, 3.],
    [1., NA, NA],
    [NA, NA, NA],
    [NA, 6.5, 3.]])
print(data)

print("--------")
# Removes all rows with at least a NaN
cleaned = data.dropna()
print(cleaned)
```

Console Shell

```
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  6.5  3.0
--------
     0    1    2
0  1.0  6.5  3.0
```

Filtering out missing data - df

main.py

```
import pandas as pd
from numpy import nan as NA

data = pd.DataFrame(
    [[1., 6.5, 3.],
    [1., NA, NA],
    [NA, NA, NA],
    [NA, 6.5, 3.]])
print(data)

print("--------")
# Removes all rows that contain ONLY NaN
cleaned = data.dropna(how = "all")
print(cleaned)
```

Console Shell

```
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  6.5  3.0
--------
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
3  NaN  6.5  3.0
```

Dropping columns

main.py ×

```
import pandas as pd
from numpy import nan as NA

data = pd.DataFrame(
        [[1., 6.5, 3.],
        [1., NA, NA],
        [NA, NA, NA],
        [NA, 6.5, 3.]])
# Adds a new column
data[4] = NA
print(data)

print("--------")
# Removes all columns with ONLY NaN
cleaned = data.dropna(axis=1, how = "all")
print(cleaned)
```

Console Shell

```
     0    1    2   4
0  1.0  6.5  3.0 NaN
1  1.0  NaN  NaN NaN
2  NaN  NaN  NaN NaN
3  NaN  6.5  3.0 NaN
--------
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  6.5  3.0
```

Fill NaN with a value

main.py

```
import pandas as pd
from numpy import nan as NA

data = pd.DataFrame(
        [[1., 6.5, 3.],
        [1., NA, NA],
        [NA, NA, NA],
        [NA, 6.5, 3.]])
print(data)

print("--------")
# Fill NaN with 0
data = data.fillna(0)
print(data)
```

Console Shell

```
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  6.5  3.0
--------
     0    1    2
0  1.0  6.5  3.0
1  1.0  0.0  0.0
2  0.0  0.0  0.0
3  0.0  6.5  3.0
```

Dataset flights.csv

- On-time data for all flights that departed NYC (i.e. JFK, LGA or EWR) in 2013

- https://data.world/bob-wakefield/flights

- And on our github

Dataset flights.csv

main.py ×

```
import pandas as pd

# https://data.world/bob-wakefield/flights
df = pd.read_csv("flights.csv")

print(df.dtypes)
```

Console Shell

```
year           int64
month          int64
day            int64
dep_time     float64
dep_delay    float64
arr_time     float64
arr_delay    float64
carrier       object
tailnum       object
flight         int64
origin        object
dest          object
air_time     float64
distance       int64
hour         float64
minute       float64
dtype: object
```

Hands-on exercises

- ✓ Change the order of the columns
- ✓ Count the NaN rows
- ✓ Drop the NaN rows
- ✓ Drop the NaN columns
- ✓ Practice with the parameter how='all'

Data Selection

Data Frames: Slicing

- There are a number of ways to subset the Data Frame:
  - one or more columns
  - one or more rows
  - a subset of rows and columns

- Rows and columns can be selected by their position or label

Slicing one column

- When selecting one column, it is possible to use single set of brackets, but the resulting object will be a Series (not a DataFrame):

```
#Select column Berserk:
df[‘Berserk']
```

Slicing one column

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df["Berserk"])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
2011    2.7
2012    2.1
2013    1.8
Name: Berserk, dtype: float64
```

Slicing one column

- A column in a DataFrame can be retrieved as a Series by dict-like notation or as attribute

- Series index and name have been kept/set appropriately

```
#Select column Berserk:
df.Berserk
```

Slicing one column

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df.Berserk)
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
2011    2.7
2012    2.1
2013    1.8
Name: Berserk, dtype: float64
```

Slicing more columns

- When we need to select more than one column and/or make the output to be a DataFrame, we should use double brackets:

```
#Select columns Berserk and One Piece:
df[["Berserk", "One Piece"]]
```

Slicing more columns

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df[["Berserk", "One Piece"]])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      Berserk  One Piece
2011      2.7        8.4
2012      2.1        9.9
2013      1.8       11.3
```

Selecting rows

- If we need to select a range of rows, we can specify the range using ":"

```
#Select the first 10 rows:
df[0:10]
```

- Notice that the first row has a position 0, and the last value in the range is omitted:
  - So for 0:10 range the first 10 rows are returned with the positions starting with 0 and ending with 9

Selecting rows

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df[0:2])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
```

Hands-on exercises

- ✓ Load flights.csv dataset
- ✓ Print only the columns relative to time and date of the flight
- ✓ Print rows 20 to 40
- ✓ Check the average *air_time* of rows 20-40
- ✓ Get the description of *distance* of the rows 100-200

Data Frames: method loc

- If we need to select a range of rows, for some columns (using their labels) we can use method loc:

```
# Select rows and cols by their label
df.loc[0:10, "day"]
```

loc

main.py ×

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df.loc[2011, "Berserk"])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
2.7
```

loc – ranges

main.py ×

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df.loc[2011:2012, "One Piece":"Berserk"])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
```

loc – lists

main.py ×

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df.loc[[2011, 2013], ["One Piece","Berserk"]])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  Berserk
2011        8.4      2.7
2013       11.3      1.8
```

Data Frames: method iloc

- If we need to select a range of rows and/or columns, using their positions we can use method iloc:

```
# Select rows and cols by their index
df.iloc[4, 2]
```

iloc

main.py ×

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df.iloc[1, 2])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
2.1
```

iloc – ranges

main.py ×

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df.iloc[:2, 0:3])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
```

iloc – lists

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
print(df.iloc[[0,2], [0,2]])
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  Berserk
2011        8.4      2.7
2013       11.3      1.8
```

iloc – summary

main.py ×

```
df.iloc[0]  # First row of  a data frame
df.iloc[i]  #(i+1)th row
df.iloc[-1] # Last row

df.iloc[:, 0]  # First column
df.iloc[:, -1] # Last column



df.iloc[0:7]       #First 7 rows
df.iloc[:, 0:2]    #First 2 columns
df.iloc[1:3, 0:2]  #Second through third rows and first 2 columns
df.iloc[[0,5], [1,3]]  #1st and 6th rows and 2nd and 4th columns
```

Data Frames: modifying columns

- We can assign values to a whole column
- It can be used to create a new column as well

- We already know a way to ”delete” one column
  - We will see later on a better way to do it

Modify a column

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
df["One Piece"] = [9, 10, 11]
print(df)
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk
2011          9            NaN      2.7
2012         10            2.3      2.1
2013         11            2.7      1.8
```

Create a column

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
df[”Naruto"] = [9, 10, 11]
print(df)
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk  Naruto
2011        8.4            NaN      2.7       9
2012        9.9            2.3      2.1      10
2013       11.3            2.7      1.8      11
```

Create a column

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
df["Naruto"] = df["One Piece"] -0.5
print(df)
```

Console Shell

```
 One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk  Naruto
2011        8.4            NaN      2.7     7.9
2012        9.9            2.3      2.1     9.4
2013       11.3            2.7      1.8    10.8
```

“Remove” a column

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
df = df[["One Piece", "One-Punch Man"]]
print(df)
```

Console Shell

```
 One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man
2011        8.4            NaN
2012        9.9            2.3
2013       11.3            2.7
```

Data Frames: modifying rows

- We can assign values to a whole row
- It can be used to create a new row as well

- We already know a way to ”delete” one row
  - We will see later on a better way to do it

Modify a row

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
df.loc[2011] = [9,1,3]
print(df)
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk
2011        9.0            1.0      3.0
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
```

Create a row

main.py

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
df.loc[2014] = [13,3.4,1.9]
print(df)
```

Console Shell

```
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
2014       13.0            3.4      1.9
```

Create a row

main.py ×

```
import pandas as pd

manga = {'One Piece': {2011: 8.4,
        2012: 9.9, 2013: 11.3},
        'One-Punch Man': {2012: 2.3, 2013: 2.7},
        'Berserk': {2011: 2.7,
        2012: 2.1, 2013: 1.8}}

df = pd.DataFrame(manga)
print(df)

print("-------")
df.loc[2014] = df.loc[2013] * 1.2
print(df)
```

Console Shell

```
 One Piece  One-Punch Man  Berserk One Piece
One-Punch Man  Berserk
2011        8.4            NaN      2.7
2012        9.9            2.3      2.1
2013       11.3            2.7      1.8
-------
      One Piece  One-Punch Man  Berserk
2011       8.40            NaN     2.70
2012       9.90           2.30     2.10
2013      11.30           2.70     1.80
2014      13.56           3.24     2.16
```

Dataset salaries.csv

- Data on academics salary.
- Columns:
  - Rank
  - Discipline
  - Years since PhD
  - Years of service
  - Sex
  - Salary

Dataset salaries.csv

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

print(df.dtypes)
```

Console Shell

```
rank           object
discipline     object
phd             int64
service         int64
sex            object
salary          int64
dtype: object
```

Hands-on exercises

- ✓Load salaries.csv dataset
- ✓Print rows 20 to 30 for *phd* and *service* columns
- ✓Print only even rows
- ✓Create a new column *pension*
- ✓Assign to the new column the *salary* multiply by the number of years of service (*service*) divided by 10
- ✓Remove the sex column
- ✓Check the average salary for Male and Females

Group By

Groupby method

The *groupby* method allows to group the rows of your data frame and extract some statistics.

Using "group by" method we can:

- Split the data into groups based on some criteria
- Calculate statistics (or apply a function) to each group

Groupby method

We use the *salaries.csv* dataset. We can group the rows based on the “rank” column.

```
# Group by the rank column
df_rank = df.groupby('rank')
```

This creates an object of type *DataFrameGroupBy.*

Groupby method

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

df_rank = df.groupby('rank')

print(df_rank)
```

Console Shell

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7f67b02f6b80>
```

Extract statistics

Once groupby object is created we can calculate various statistics for each group. Eg.

```
# Group by the rank column
print(df_rank.mean())
print(df_rank.max())
```

Extract statistics

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

df_rank = df.groupby('rank')

print(df_rank.mean())
print("_______")
print(df_rank.max())
```

Console Shell

```
                 phd    service         salary
rank
AssocProf  15.076923  11.307692   91786.230769
AsstProf    5.052632   2.210526   81362.789474
Prof       27.065217  21.413043  123624.804348
_______
          discipline  phd  service   sex  salary
rank
AssocProf          B   26       24  Male  119800
AsstProf           B   11        6  Male   97032
Prof               B   56       51  Male  186960
```

Extract statistics one-column

It is possible to extract a single statistic for a specific column.

```
# Group by “rank” and display ‘salary’ min
df.groupby('rank')['salary'].min()
```

Extract statistics one-column

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

print(df.groupby('rank')['salary'].mean())
print("_______")
print(df.groupby('rank')['salary'].max())
print("_______")
print(df.groupby('rank')['salary'].min())
```

Console Shell

```
rank
AssocProf     91786.230769
AsstProf      81362.789474
Prof         123624.804348
Name: salary, dtype: float64
_______
rank
AssocProf    119800
AsstProf      97032
Prof         186960
Name: salary, dtype: int64
_______
rank
AssocProf    62884
AsstProf     63100
Prof         57800
Name: salary, dtype: int64
```

Extract statistics multiple columns

It is possible to extract a single statistic for a specific group of columns.

```
# Group by “rank” and display ‘salary’ and
#  ’service’ min
df.groupby('rank')[['salary', 'service']].min()
```

Extract statistics multiple columns

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

print(df.groupby('rank')[['salary', 'service']].mean())
print("_______")
print(df.groupby('rank')[['salary', 'service']].max())
print("_______")
print(df.groupby('rank')[['salary', 'service']].min())
```

Console Shell

```
 salary    service
rank
AssocProf   91786.230769  11.307692
AsstProf    81362.789474   2.210526
Prof       123624.804348  21.413043
_______
           salary  service
rank
AssocProf  119800       24
AsstProf    97032        6
Prof       186960       51
_______
           salary  service
rank
AssocProf   62884        6
AsstProf    63100        0
Prof        57800        0
```

Groupby multiple columns

Groupby can be used for grouping several columns

```
# Calculate mean salary for each professor
# rank for each gender
df.groupby(['rank’,’sex’])[['salary']].mean()
```

Groupby multiple columns

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

print(df.groupby(['rank', 'sex'])['salary'].mean())
```

Console Shell

```
            rank       sex
AssocProf   Female     88512.800000
            Male      102697.666667
AsstProf    Female     78049.909091
            Male       85918.000000
Prof        Female    121967.611111
            Male      124690.142857
Name: salary, dtype: float64
```

Which statistics?

The statistics we can extract from a groupby are:

- Total sum

```
df.groupby('rank')['salary’].sum()
```

- Max and min

```
df.groupby('rank')['salary’].max()
df.groupby('rank')['salary’].min()
```

- Count number of elements

```
df.groupby('rank')['salary’].count()
```

- Average

```
df.groupby('rank')['salary’].mean()
```

- Median

```
df.groupby('rank')['salary’].median()
```

Group by: Missing Values

- When summing the data, missing values will be treated as zero
- If all values are missing, the sum will be equal to NaN
- Missing values in GroupBy method are excluded
- Many descriptive statistics methods have skipna option to control if missing data should be excluded. This value is set to True by default

Hands-on exercises

- ✓Load coffee.csv dataset
- ✓Check the average coding time (‘Coding Hours”) based on the favourite coffee (‘CoffeeType’)
- ✓Check the maximum number of coffee cups (‘CoffeeCupsPerDay’) for each “AgeRange”
- ✓Group the rows of the data frame by their class (“CoffeeType”) and print the mean cups per day(“CoffeeCupsPerDay”)
- ✓Group the rows by their “CoffeeType” and then print the average value of all the numeric columns

Sorting

- We can sort the data by a value in the column. By default the sorting will occur in ascending order and a new data frame is return.

```
# Create a new data frame from the original
# sorted by the column Salary

df_sorted = df.sort_values( by ='salary')
```

Sorting

main.py ×

```
import pandas as pd


df = pd.read_csv("salaries.csv")


# Create a new data frame from the original sorted by the column Salary
df_sorted = df.sort_values(by ='salary')
print(df_sorted.head())
```

Console Shell

```
         rank discipline  phd  service     sex  salary
9        Prof          A   51       51    Male   57800
54  AssocProf          A   25       22  Female   62884
66   AsstProf          A    7        6  Female   63100
71  AssocProf          B   12        9  Female   71065
57   AsstProf          A    3        1  Female   72500
```

Sorting - decreasing

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

# Create a new data frame from the original sorted by the column Salary
df_sorted = df.sort_values(by ='salary', ascending = False)
print(df_sorted.head())
```

Console Shell

```
    rank discipline  phd  service     sex  salary
0   Prof          B   56       49    Male  186960
13  Prof          B   35       33    Male  162200
72  Prof          B   24       15  Female  161101
27  Prof          A   45       43    Male  155865
31  Prof          B   22       21    Male  155750
```

Sorting – multiple columns

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

# Sorting by 'service', ties are resolved using the higher salary first
df_sorted = df.sort_values(by =['service', 'salary'],
                           ascending = [True, False])
print(df_sorted.head(10))
```

Console Shell

```
        rank discipline  phd  service     sex  salary
52      Prof          A   12        0  Female  105000
17  AsstProf          B    4        0    Male   92000
12  AsstProf          B    1        0    Male   88000
23  AsstProf          A    2        0    Male   85000
43  AsstProf          B    5        0  Female   77000
55  AsstProf          A    2        0  Female   72500
57  AsstProf          A    3        1  Female   72500
28  AsstProf          B    7        2    Male   91300
42  AsstProf          B    4        2  Female   80225
68  AsstProf          A    4        2  Female   77500
```

Hands-on exercises

- ✓Load coffee.csv dataset
- ✓Create a new data frame df_sorted that contains all the answers sorted by the number of cups of coffee per day(“CoffeeCupsPerDay”) in increasing order (smaller above).
- ✓Print the row with the maximum “CoffeeCupsPerDay”.
- ✓Which type of coffee is this person drinking?

Aggregation function

- Aggregation - computing a summary statistic about each group, i.e.
  - compute group sums or means
  - compute group sizes/counts

```
# Calculate min, mean  and max for ‘salary’ and
# ‘service’
df[['salary','service']].agg(['min', 'mean', 'max'])
```

Aggregate

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

print(df[['salary','service']].agg(['min', 'mean', 'max']))
```

Console Shell

```
              salary    service
min    57800.000000   0.000000
mean  108023.782051  15.051282
max   186960.000000  51.000000
```

Aggregation function

Common aggregation functions:

- min, max
- count, sum, prod
- mean, median

Groupby + Aggregation

Aggregation can be used after a groupby.

```
# Group by ‘sex’ and display max and mean ‘salary’
df.groupby('sex')['salary'].agg(['max', 'mean'])
```

Groupby + Aggregation

main.py ×

```
import pandas as pd

df = pd.read_csv("salaries.csv")

# Group by 'sex' and display max and mean 'salary'
print(df.groupby('sex')['salary'].agg(['max', 'mean']))
```

Console Shell

```
            max           mean
sex
Female   161101  101002.410256
Male     186960  115045.153846
```

Hands-on exercises

Have a look to the table Salaries and think in conclusions that could be extracted from this data and that would be interesting from the point of view of Digital Humanities. Write the code associated by using Pandas for obtaining these conclusions!

That's all Folks!
SUPERB wallpapers